

LOW LEVEL DESIGN DOCUMENT

# CredVault

Detailed architectural blueprint — class designs, database schemas, API specifications, message contracts, security implementations, and testing approaches.

Angular 19

.NET 10

Ocelot Gateway

RabbitMQ + MassTransit

SQL Server

BCrypt · JWT · OTP

CQRS + MediatR

Clean Architecture

DOCUMENT TYPE

LLD

VERSION

2.0

SERVICES

5 Core

YEAR

2026

# Table of Contents

CredVault Low Level Design · 11 Sections + Appendices

|    |                                      |       |
|----|--------------------------------------|-------|
| 01 | Introduction                         | ..... |
| 02 | Identity Service                     | ..... |
| 03 | Card Service                         | ..... |
| 04 | Billing Service                      | ..... |
| 05 | Payment Service                      | ..... |
| 06 | Notification Service                 | ..... |
| 07 | API Gateway & Endpoint Specification | ..... |
| 08 | Database Schemas                     | ..... |
| 09 | Message Contracts                    | ..... |
| 10 | Frontend Component Tree              | ..... |
| 11 | Error Handling & Testing             | ..... |
| A  | Naming Conventions                   | ..... |
| B  | Git Workflow                         | ..... |
| C  | CI/CD Pipeline                       | ..... |

# Introduction

This Low-Level Design document provides **implementation-ready technical specifications** for the CredVault platform. It translates high-level architectural patterns into concrete class designs, schema definitions, API contracts, and code-level details — ensuring consistency across all five core microservices.

## Document Scope



### Class Designs

Domain Entities, CQRS Handlers, Validators across all Clean Architecture layers.



### DB Schemas

5 service-specific SQL Server databases with full DDL definitions.



### API Specification

Ocelot Gateway routing and every service endpoint with method, auth, and description.



### Message Contracts

MassTransit events and saga state machine definitions for async communication.



### Frontend

Angular 19 standalone component hierarchy mirroring the service domain boundaries.



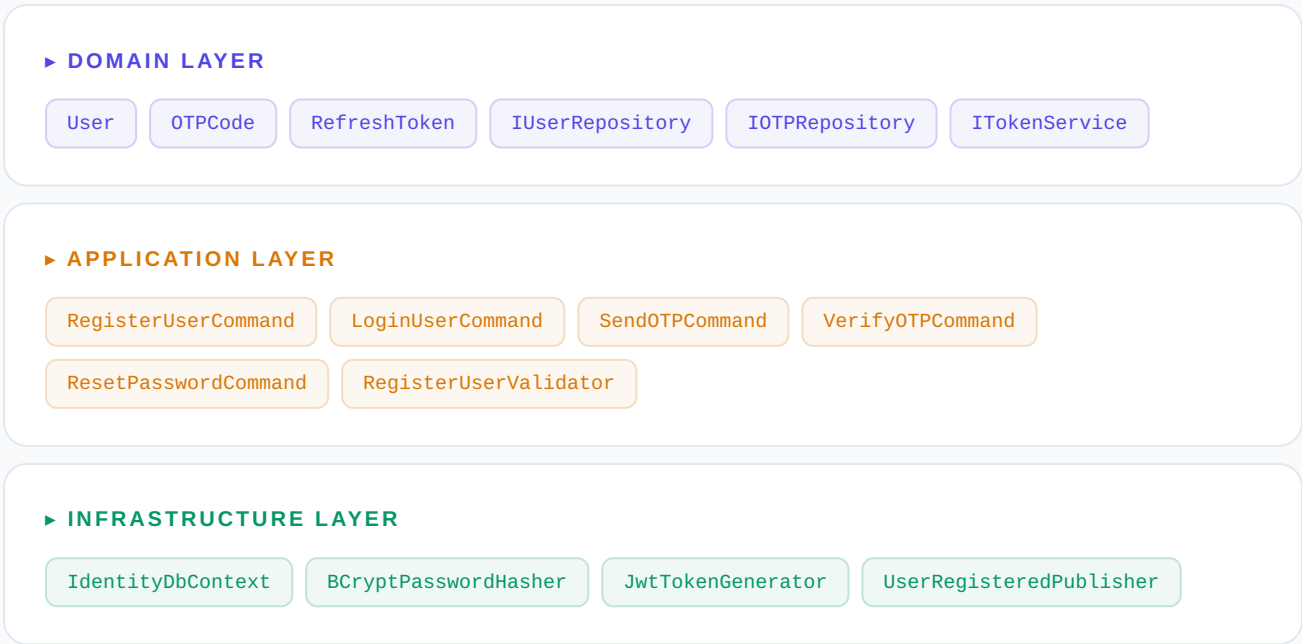
### Security

JWT · BCrypt · OTP · CORS · Token validation implementation details.

# Identity Service

|                                  |                                                   |                                                                      |
|----------------------------------|---------------------------------------------------|----------------------------------------------------------------------|
| <div>PORT</div> <div>:5032</div> | <div>DATABASE</div> <div>credvault_identity</div> | <div>RESPONSIBILITY</div> <div>Auth · OTP · Profile · Security</div> |
|----------------------------------|---------------------------------------------------|----------------------------------------------------------------------|

## Architecture Layers



## OTP Lifecycle — Sequence Diagram

| # | From     | To               | Action          | Detail                            |
|---|----------|------------------|-----------------|-----------------------------------|
| 1 | Client   | Identity Service | SendOTP Request | User triggers OTP (Login / Reset) |
| 2 | Identity | Internal         | Generate Code   | 6-digit numeric string            |
| 3 | Identity | Internal         | Hash Code       | BCrypt hashing — Work Factor 12   |
| 4 | Identity | Database         | Save OTP        | Store Hash + Expiry (5 min TTL)   |
| 5 | Identity | Notification Svc | Publish Event   | OTPSentEvent → email delivery     |
| 6 | Client   | Identity Service | VerifyOTP       | User submits raw 6-digit code     |
| 7 | Identity | Internal         | Verify Hash     | Compare input vs stored hash      |
| 8 | Identity | Database         | Mark Used       | Set IsUsed = true                 |

## Key Implementation — JWT Generation



```
public string GenerateAccessToken(User user)
{
    var claims = new List<Claim>
    {
        new Claim(JwtRegisteredClaimNames.Sub, user.Id.ToString()),
        new Claim(JwtRegisteredClaimNames.Email, user.Email),
        new Claim(ClaimTypes.Role, user.Role),
        new Claim("name", user.FullName)
    };

    var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_config["Jwt:Secret"]));
    var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

    var token = new JwtSecurityToken(
        issuer: _config["Jwt:Issuer"],
        audience: _config["Jwt:Audience"],
        claims: claims,
        expires: DateTime.UtcNow.AddMinutes(15),
        signingCredentials: creds);

    return new JwtSecurityTokenHandler().WriteToken(token);
}
```

# Card Service

PORT

:5033

DATABASE

credvault\_cards

RESPONSIBILITY

Card Vaulting · Issuer  
Detection

## Card Masking Pipeline

### ADD CARD REQUEST PIPELINE

- 1 **Input:** Raw card number received — e.g. "1234567812345678"
- 2 **Detect Issuer:** BIN prefix check — 4... = Visa, 5... = MasterCard, 37... = Amex
- 3 **Deduplication Hash:** SHA-256 hash of the full card number to prevent duplicate vaulting
- 4 **Mask Number:** Store only last 4 digits → "\*\*\*\* \* 5678"
- 5 **Persist:** (MaskedNumber, IssuerId, CardNumberHash, CreditLimit) written to CardDB
- 6 **Return:** CardResponse with masked number — raw PAN is never stored or returned

## Card Domain Entity

```
C# — CREDITCARD.CS

public static CreditCard Create(Guid userId, string cardNumber, ...)
{
    return new CreditCard
    {
        Id = Guid.NewGuid(),
        UserId = userId,
        MaskedNumber = $"**** * {cardNumber.Substring(cardNumber.Length - 4)}",
        CardNumberHash = HashService.ComputeSha256(cardNumber),
        CreatedAt = DateTime.UtcNow
    };
}

public void UpdateOutstandingBalance(decimal amount)
{
    OutstandingBalance = Math.Max(0, OutstandingBalance + amount);
    UpdatedAt = DateTime.UtcNow;
}
```

# Billing Service

PORT

:5034

DATABASE

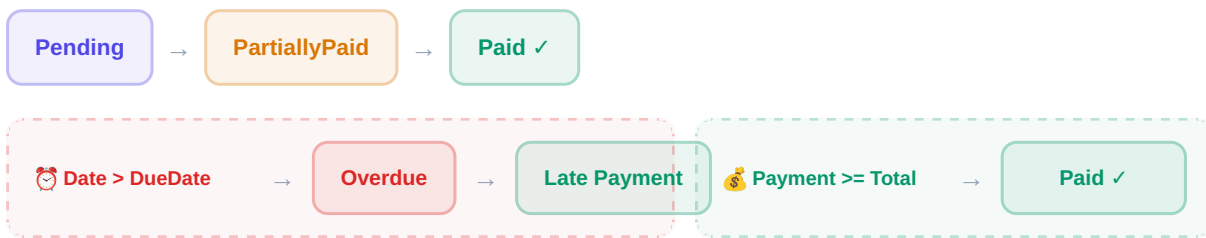
credvault\_billing

RESPONSIBILITY

Bills · Schedules ·  
Rewards

## Bill Status State Machine

### BILL LIFECYCLE — STATE TRANSITIONS



## Reward Tier System

| Tier     | Min Points | Multiplier | Benefit                                    |
|----------|------------|------------|--------------------------------------------|
| Silver   | 0          | 1×         | Base rewards on every successful payment   |
| Gold     | 1,000      | 1.5×       | Enhanced multiplier + priority support     |
| Platinum | 5,000      | 2.5×       | Max multiplier, cashback, concierge access |

## Implementation — Apply Payment

```
public void ApplyPayment(decimal amount)
{
    AmountPaid += amount;
    Status      = AmountPaid >= TotalAmount ? "Paid" : "PartiallyPaid";
    UpdatedAt   = DateTime.UtcNow;
}
```

# Payment Service

PORT

:5035

DATABASE

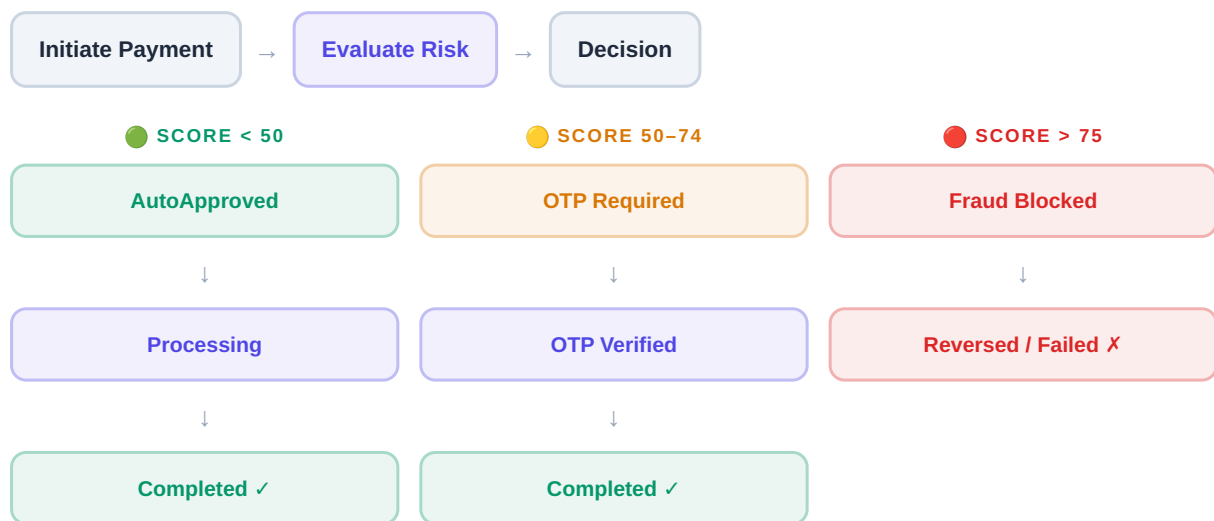
credvault\_payments

RESPONSIBILITY

Saga Orchestration ·  
Risk Score

## Payment Saga State Machine

### PAYMENT FLOW — MASSTRANSIT SAGA STATE TRANSITIONS



## Risk Engine Implementation

```
C# — RISKENGINE.CS

public int EvaluateRisk(decimal amount, Guid userId)
{
    int score = 0;
    if (amount > 50000) score += 40; // High-value transaction
    if (IsUnusualTime()) score += 20; // 1 AM - 4 AM window
    if (HasRecentFailures(userId)) score += 20; // Prior failed attempts
    return score;
}
```

# Notification Service

PORT

**:5036**

DATABASE

**credvault\_notifications**

RESPONSIBILITY

**Event Consumption** ·  
**Email Delivery**

## Event Processing Pipeline

EVENT → EMAIL FLOW

1

**RabbitMQ Event Consumed** — e.g. `PaymentCompletedEvent`, `OTPSentEvent`

2

**Extract Recipient** — User email resolved from event payload (no inter-service HTTP call)

3

**Fetch Template** — HTML email template retrieved by event key from the notifications DB

4

**Render HTML** — Replace `{{placeholders}}` with event payload values (name, amount, OTP, etc.)

5

**Send via SendGrid** — Dispatched with retry policy. Failed sends are written to the DLQ

6

**Audit Log** — Notification result (success/failure) persisted to the notifications DB for traceability

# API Gateway & Endpoint Specification

The **Ocelot API Gateway** manages all cross-cutting concerns — JWT Bearer validation, rate limiting, and request routing. All endpoints below are exposed through the gateway; services are never directly accessible from outside the Docker network.

## 7.1 — Identity Service :5032

| Path                                  | Method | Auth | Description                                      |
|---------------------------------------|--------|------|--------------------------------------------------|
| <code>/api/auth/register</code>       | POST   | No   | Create user account and fire welcome event       |
| <code>/api/auth/login</code>          | POST   | No   | Validate credentials; return JWT + Refresh Token |
| <code>/api/auth/refresh</code>        | POST   | No   | Issue new JWT using valid Refresh Token          |
| <code>/api/auth/verify-email</code>   | POST   | No   | Verify account via registration OTP              |
| <code>/api/auth/send-otp</code>       | POST   | No   | Trigger OTP for login, payment, or reset         |
| <code>/api/auth/verify-otp</code>     | POST   | No   | Validate a 6-digit numeric OTP                   |
| <code>/api/auth/reset-password</code> | POST   | No   | Set new password using verified OTP              |
| <code>/api/users/profile</code>       | GET    | JWT  | Retrieve current user profile details            |
| <code>/api/users/profile</code>       | PUT    | JWT  | Update user profile — Name, Phone                |

## 7.2 — Card Service :5033

| Path                                 | Method | Auth | Description                                        |
|--------------------------------------|--------|------|----------------------------------------------------|
| <code>/api/cards</code>              | GET    | JWT  | List all cards associated with the user            |
| <code>/api/cards/{id}</code>         | GET    | JWT  | Retrieve specific card metadata                    |
| <code>/api/cards/add</code>          | POST   | JWT  | Securely vault a new card — masking + SHA-256 hash |
| <code>/api/cards/{id}</code>         | DELETE | JWT  | Soft-delete a card from the vault                  |
| <code>/api/cards/default/{id}</code> | PUT    | JWT  | Set card as primary payment method                 |
| <code>/api/cards/verify</code>       | POST   | JWT  | Execute micro-auth (₹1) to verify card is active   |
| <code>/api/cards/utilization</code>  | GET    | JWT  | Calculate credit utilization percentage            |

### 7.3 — Billing Service :5034

| Path                                  | Method | Auth | Description                               |
|---------------------------------------|--------|------|-------------------------------------------|
| <code>/api/bills</code>               | GET    | JWT  | Retrieve paged history of monthly bills   |
| <code>/api/bills/{id}</code>          | GET    | JWT  | Get line-item details for a specific bill |
| <code>/api/bills/schedule</code>      | POST   | JWT  | Schedule a future payment for a bill      |
| <code>/api/bills/schedule/{id}</code> | DELETE | JWT  | Cancel a pending scheduled payment        |
| <code>/api/rewards</code>             | GET    | JWT  | Get points balance, tier, and history     |
| <code>/api/rewards/redeem</code>      | POST   | JWT  | Convert points to cashback or vouchers    |

### 7.4 — Payment Service :5035

| Path                                       | Method | Auth | Description                             |
|--------------------------------------------|--------|------|-----------------------------------------|
| <code>/api/payments/initiate</code>        | POST   | JWT  | Trigger the Payment Saga state machine  |
| <code>/api/payments/history</code>         | GET    | JWT  | Retrieve paged list of all transactions |
| <code>/api/payments/{id}</code>            | GET    | JWT  | Check real-time status of a payment     |
| <code>/api/payments/verify-otp</code>      | POST   | JWT  | Validate OTP for high-risk transactions |
| <code>/api/payments/risk-score/{id}</code> | GET    | JWT  | View full risk assessment breakdown     |
| <code>/api/payment-methods</code>          | GET    | JWT  | List available payment gateway channels |

### 7.5 — Notification Service :5036

| Path                                  | Method | Auth  | Description                              |
|---------------------------------------|--------|-------|------------------------------------------|
| <code>/api/notifications/logs</code>  | GET    | JWT   | Retrieve history of emails sent to user  |
| <code>/api/notifications/audit</code> | GET    | Admin | System-wide audit log — Admin restricted |

# Database Schemas

## Identity DB — credvault\_identity

```
CREATE TABLE Users (  
  Id          UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),  
  Email       NVARCHAR(256)    UNIQUE NOT NULL,  
  PasswordHash NVARCHAR(512)   NOT NULL,  
  FullName    NVARCHAR(100)    NOT NULL,  
  Role        NVARCHAR(20)     DEFAULT 'User',  
  IsEmailVerified BIT         DEFAULT 0,  
  CreatedAt   DATETIMEOFFSET   DEFAULT GETUTCDATE()  
);
```

## Payment DB — credvault\_payments

```
CREATE TABLE Payments (  
  Id          UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),  
  UserId      UNIQUEIDENTIFIER NOT NULL,  
  BillId      UNIQUEIDENTIFIER NOT NULL,  
  Amount      DECIMAL(18,2)    NOT NULL,  
  Status      NVARCHAR(20)     DEFAULT 'Processing',  
  CreatedAt   DATETIMEOFFSET   DEFAULT GETUTCDATE()  
);
```

# Message Contracts

All inter-service communication uses **RabbitMQ via MassTransit**. Events are defined as immutable C# records with a `CorrelationId` for end-to-end traceability across services.



C# — EVENTS.CS

*// Published by Payment Service → consumed by Billing + Notification*

```
public record PaymentCompletedEvent(  
    Guid    PaymentId,  
    Guid    UserId,  
    Guid    BillId,  
    decimal Amount,  
    Guid    CorrelationId,  
    DateTime Timestamp  
);
```

*// Published by Billing Service → consumed by Notification*

```
public record RewardEarnedEvent(  
    Guid    UserId,  
    int     Points,  
    string  Tier,  
    Guid    CorrelationId  
);
```

# Frontend Component Tree

Angular 19 standalone application. Component hierarchy is structured to mirror domain service boundaries — each feature module maps directly to one backend microservice.

```
AppComponent └─ ShellComponent ── Navbar (Auth state, Notification bell, User avatar)
└─ RouterOutlet ── DashboardComponent ── CardsModule ── CardListComponent ──
AddCardComponent ── BillingModule ── BillDetailComponent ──
RewardsSummaryComponent ── PaymentModule ── InitiatePaymentComponent ──
PaymentStatusComponent
```

# Error Handling & Testing

## HTTP Status Code Mapping

404

**NotFoundException**

Returned when a requested resource (User, Card, Bill, Payment) does not exist or belongs to another user.

400

**ValidationException**

FluentValidation failure — invalid card number format, expired OTP, missing required fields, or business rule violation.

401

**UnauthorizedException**

JWT is missing, expired, or has an invalid signature. Client must re-authenticate or use the refresh token endpoint.

403

**FraudException**

Risk score exceeded the block threshold (> 75). Transaction permanently declined and a FraudDetected event is published.

## Testing Strategy

**Unit Tests**

xUnit for all Domain Entities and CQRS Command Handlers. Coverage target > 80%.

**Integration Tests**

Testing MassTransit consumers and EF Core repositories against an in-memory or containerized DB.

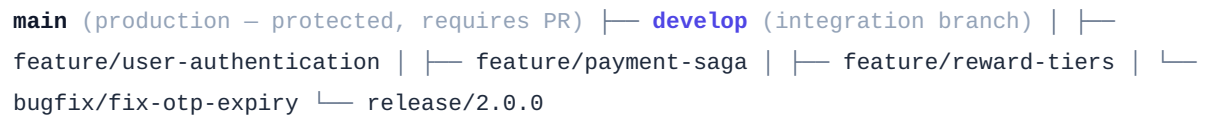
**E2E Tests**

Playwright for critical payment journeys — login → card add → bill pay → receipt verification.

# Naming Conventions

| Element    | Convention                           | Example                                                   |
|------------|--------------------------------------|-----------------------------------------------------------|
| Projects   | <code>{ServiceName}.{Layer}</code>   | <code>PaymentService.Application</code>                   |
| Commands   | <code>{Action}{Entity}Command</code> | <code>CreateBillCommand</code>                            |
| Queries    | <code>{Action}{Entity}Query</code>   | <code>GetBillsByUserIdQuery</code>                        |
| Events     | <code>{Entity}{Action}Event</code>   | <code>PaymentCompletedEvent</code>                        |
| DTOs       | <code>{Entity}{Type}</code>          | <code>BillResponse</code> , <code>LoginRequest</code>     |
| Interfaces | <code>I{ServiceName}</code>          | <code>IService</code>                                     |
| Entities   | Singular noun                        | <code>User</code> , <code>Card</code> , <code>Bill</code> |

# Git Workflow



```
graph TD
    main["main (production – protected, requires PR)"]
    develop["develop (integration branch)"]
    feature1["feature/user-authentication"]
    feature2["feature/payment-saga"]
    feature3["feature/reward-tiers"]
    bugfix["bugfix/fix-otp-expiry"]
    release["release/2.0.0"]

    main --> develop
    develop --> feature1
    develop --> feature2
    develop --> feature3
    feature1 --> develop
    feature2 --> develop
    feature3 --> develop
    develop --> bugfix
    bugfix --> main
    bugfix --> release
    release --> main
```

**main** (production – protected, requires PR) | — **develop** (integration branch) | —  
feature/user-authentication | — feature/payment-saga | — feature/reward-tiers | —  
bugfix/fix-otp-expiry — release/2.0.0

# CI/CD Pipeline

## DEPLOYMENT PIPELINE — GITHUB ACTIONS

- 1 Push to GitHub** — Developer pushes to a feature branch and opens a Pull Request to develop
- 2 Run Tests** — xUnit unit tests + integration tests execute automatically. Failure blocks merge and notifies team
- 3 Build Docker Images** — Each microservice is containerized with its own `Dockerfile` via Docker Compose
- 4 Push to Registry** — Tagged images pushed to the Container Registry (e.g. GitHub Container Registry)
- 5 Deploy to Staging** — Automated deployment to staging environment for QA validation
- 6 Run Integration Tests** — Playwright E2E suite runs against the staging environment
- 7 Deploy to Production** 🚀 — Manual approval gate before promoting to production

## CredVault — Low Level Design Document

CredVault · v2.0 · May 2026

This document is confidential and intended for internal review only.